

AquaWell Water Refill Station System

Project Documentation, File Inventory, Dependencies, Progress & Source-Derived Database Schema

Snapshot: October 4, 2026 **Origin:** Built from scratch from Git initial commit ff0bf40

Executive Summary

AquaWell is a React/TypeScript web application for water-refilling-station operations. The current codebase includes public/authentication pages, customer ordering, staff/delivery workflows, an extensive admin dashboard, Supabase integration, inventory, sales, notifications, mapping and demand forecasting.

Current status: advanced prototype / near-functional application, but not build-clean. The current source fails npm run build due to TypeScript errors. No SQL migration/schema files are included in the ZIP, so the database diagram is source-derived rather than a verified dump of the live PostgreSQL schema.

Development Progress

Date	Commit	Summary
Oct 1	ff0bf40	Initial commit
Oct 1	e96ccf0	Landing, auth modals, admin dashboard, theme switcher
Oct 1	619cb9d	Light/dark theme system
Oct 1	ca7e81b	Admin CRUD, demand forecasts, typography
Oct 1	74b9bcc	Validation, notifications, audit logs
Oct 1	267f6d6	HashRouter for publishing
Oct 1	6eef912	Low-stock alerts, history, validation
Oct 2	8ab95f6	Admin Supabase cloud integration
Oct 2	f32b991	Full Supabase integration across roles
Oct 2	31e44be	Realtime notifications and timestamps
Oct 2	3ca7aa9	Unused import cleanup
Oct 4	fa59e0b	Customer directory/deletion workflow fix

Technology & Dependencies

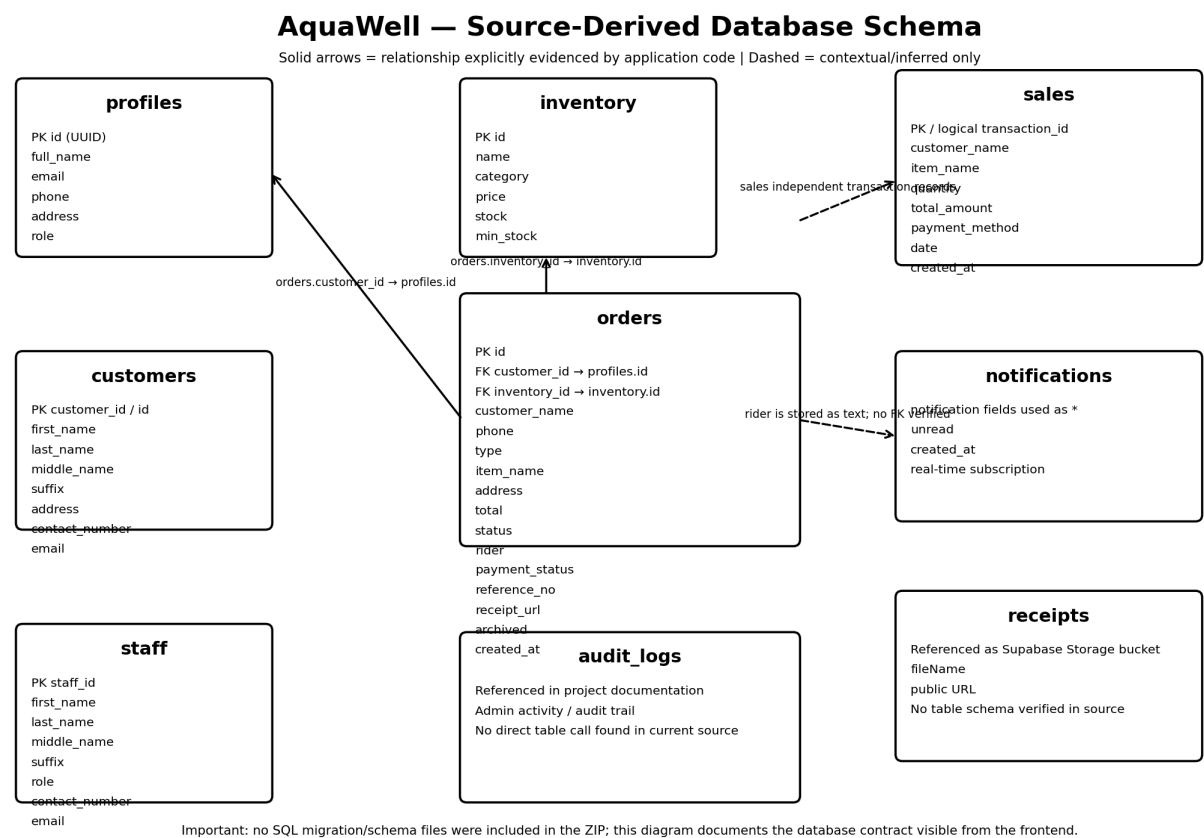
Category	Technology
Frontend	React 19, React DOM, TypeScript, Vite
Routing	react-router-dom 7
Backend	Supabase Auth + PostgreSQL + Storage
UI	Tailwind CSS/PostCSS, Lucide React, Radix primitives
Maps	Leaflet + React Leaflet
Quality	ESLint + TypeScript
Deployment tooling	GitHub Pages script; existing dist bundle

Application Modules

Module	Status / capabilities
Public/Auth	Landing, login, registration, auth modal infrastructure, password change, theme support
Admin	Orders, riders, inventory, customers, staff, sales, reports, forecasts, notifications
Customer	Profile, products, ordering, delivery/pickup, payments, receipt upload, order history
Staff/Delivery	Role lookup, orders, rider filtering, status updates, inventory, sales, mapping

Source-Derived Database Schema

Verified table references in the frontend include profiles, customers, staff, inventory, orders, sales and notifications. The customer dashboard also uses a Supabase Storage bucket named receipts. The project documentation references audit_logs, but the current source does not directly query that table.



Verified relationships: orders.inventory_id → inventory.id is explicitly described as a relational foreign key in source. orders.customer_id is populated with the authenticated Supabase user id and queried against it; this strongly indicates a relationship to profiles.id, although the actual PostgreSQL FK constraint is not present in the archive.

Key Tables

Table/resource	Observed fields / purpose
profiles	id, full_name, email, phone, address, role
customers	customer_id/id, first_name, last_name, middle_name, suffix, address, contact_number/phone, email
staff	staff_id, first_name, last_name, middle_name, suffix, role, contact_number, email
inventory	id, name, category, price, stock, min_stock
orders	id, customer_id, inventory_id, customer_name, phone, type, item_name, address, total, status, rider, payment_status, reference_no, receipt_url, archived, created_at
sales	transaction_id, customer_name, item_name, quantity, total_amount, payment_method, date, created_at
notifications	unread plus additional notification fields accessed via select(*)
receipts	Supabase Storage bucket; file name/public URL workflow

Table/resource	Observed fields / purpose
audit_logs	Referenced by project documentation; exact schema not verified in frontend source

Verification Results

npm run build: FAILED. The main structural error is App.tsx rendering without the required AuthModals props. Additional failures are unused declarations in ChangePasswordModal, AdminDashboard, CustomerDashboard and StaffDashboard. **npm run lint:** the supplied extracted environment returned “eslint: Permission denied”, so a trustworthy lint count could not be established from this run.

Recommended Next Steps

- Make npm run build pass and then perform a clean lint run.
- Move Supabase URL/key configuration to Vite environment variables.
- Add Supabase SQL migrations, RLS policies, indexes and trigger definitions to source control.
- Normalize customer/profile/staff relationships and use rider_id rather than rider names.
- Use numeric PostgreSQL money types instead of formatted currency strings.
- Split the very large AdminDashboard.tsx into smaller feature modules.
- Add automated tests for authentication, orders, inventory, payments, roles and forecasting.